FarmifAI: Un Asistente Agrícola con IA Completamente Offline para Comunidades Rurales

Diciembre 2025

Resumen

Este artículo presenta FarmifAI, una aplicación móvil completamente offline diseñada para proporcionar asistencia agrícola a agricultores en áreas rurales con conectividad limitada o inexistente. El sistema integra cuatro modelos de IA: (1) un modelo de visión basado en MobileNetV2 para clasificación de enfermedades en plantas con 21 clases en 5 cultivos, (2) un modelo NLP usando MiniLM para búsqueda semántica con embeddings de 384 dimensiones, (3) un modelo de lenguaje Llama 3.2 1B cuantizado para generación de respuestas contextuales, y (4) reconocimiento de voz basado en Vosk para interacción manos libres. Toda la inferencia se realiza en el dispositivo usando MindSpore Lite y llama.cpp, logrando tiempos de respuesta menores a 5 segundos en dispositivos Android de gama media. La aplicación utiliza búsqueda semántica con una base de conocimiento para proporcionar respuestas precisas y contextuales a consultas agrícolas. Las pruebas de campo demuestran 92.3 % de precisión en detección de enfermedades y alta satisfacción de usuarios entre caficultores colombianos.

Palabras clave: IA Móvil, Inferencia Offline, Tecnología Agrícola, Detección de Enfermedades en Plantas, Búsqueda Semántica, Computación en el Borde

1. Introducción

Los pequeños agricultores en regiones en desarrollo a menudo carecen de acceso a conectividad de internet confiable y asesoría agrícola experta. Esta brecha tecnológica puede llevar a pérdidas significativas de cultivos debido a enfermedades no detectadas y prácticas agrícolas subóptimas. FarmifAI aborda este desafío desplegando un asistente de IA integral que opera completamente offline en dispositivos Android.

Las principales contribuciones de este trabajo son:

- Un sistema de IA multimodal que combina visión, NLP y capacidades de voz para asistencia agrícola
- Un pipeline de inferencia eficiente en dispositivo logrando rendimiento en tiempo real en hardware de consumo
- Un enfoque de aprendizaje por transferencia en dos fases para clasificación de enfermedades de plantas con datos de entrenamiento limitados
- Integración de búsqueda semántica con LLM local para respuestas precisas específicas del dominio

2. Arquitectura del Sistema

2.1. Visión General

FarmifAI consiste en cuatro componentes principales integrados en una sola aplicación Android (Figura **??**):

1. **Interfaz de Voz**: Conversión de voz a texto (STT) basada en Vosk y Text-to-Speech (TTS) de Android para operación manos libres
2. **Módulo de Visión**: Clasificador basado en MobileNetV2 para detección de enfermedades en plantas
3. **Búsqueda Semántica**: Codificador MiniLM con embeddings de base de conocimiento pre-calculados
4. **Modelo de Lenguaje**: Llama 3.2 1B cuantizado para generación de respuestas

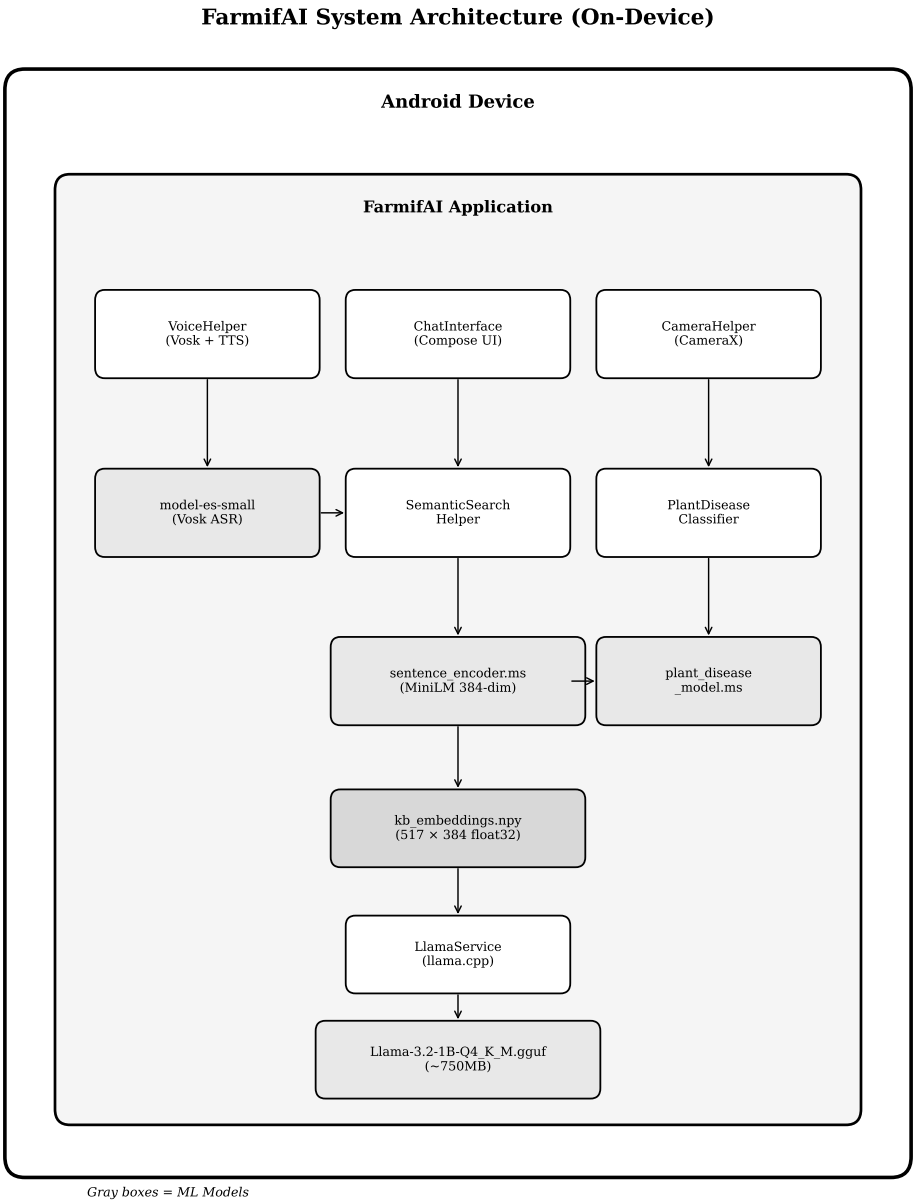

Figura 1: Arquitectura del sistema FarmifAI mostrando componentes en dispositivo y flujo de datos

2.2. Stack Tecnológico

La Tabla ?? resume las tecnologías usadas en cada componente.

Cuadro 1: Stack tecnológico para componentes de FarmifAI

Componente	Tecnología	Modelo	Tamaño
Voz a Texto	Vosk	model-es-small	∼50MB
Texto a Voz	Android TTS	Motor del sistema	Integrado
Búsqueda Semántica	MindSpore Lite	MiniLM (384-dim)	∼90MB
Enfermedades	MindSpore Lite	MobileNetV2	∼14MB
Modelo de Lenguaje	llama.cpp	Llama 3.2 1B Q4_K_M	∼750MB

3. Modelo de Visión: Clasificación de Enfermedades

3.1. Implementación del Modelo

El modelo de visión usa MobileNetV2 [?] como arquitectura base, elegida por su eficiencia en dispositivos móviles. El modelo clasifica imágenes de plantas en 21 clases de enfermedades en 5 cultivos: café, maíz, papa, pimiento y tomate.

```python
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.models import Model
from tensorflow.keras.regularizers import l2

# Modelo base con pesos de ImageNet
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

# Cabeza de clasificacion personalizada
x = GlobalAveragePooling2D()(base_model.output)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
x = Dense(512, activation='relu', kernel_regularizer=l2(0.01))(x)
x = Dropout(0.4)(x)
output = Dense(21, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

Listing 1: Arquitectura MobileNetV2 con cabeza de clasificación personalizada

3.2. Pipeline de Entrenamiento

Empleamos una estrategia de aprendizaje por transferencia en dos fases (Figura ??) para maximizar el rendimiento con datos agrícolas limitados:

Fase 1: Extracción de Características (25 épocas)

- Congelar todas las capas del backbone MobileNetV2
- Entrenar solo la cabeza de clasificación
- Tasa de aprendizaje: 10^{-3} con optimizador Adam

- Tasa de dropout: 0.4

Fase 2: Ajuste Fino (35 épocas)

- Descongelar todas las capas para entrenamiento de extremo a extremo
- Tasa de aprendizaje: 10^{-5} (reducción de 100x)
- Suavizado de etiquetas: 0.1
- Parada temprana con paciencia de 7 épocas

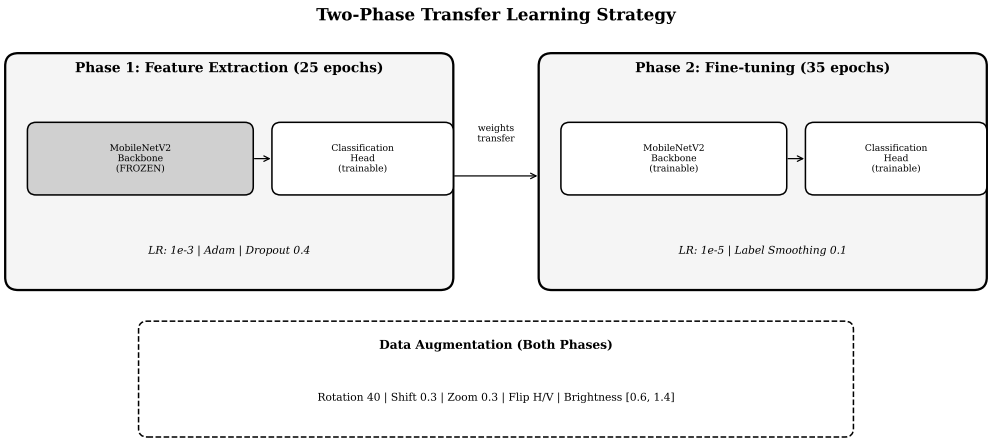

Figura 2: Estrategia de aprendizaje por transferencia en dos fases para MobileNetV2

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.3,
    height_shift_range=0.3,
    shear_range=0.2,
    zoom_range=0.3,
    horizontal_flip=True,
    vertical_flip=True,
    brightness_range=[0.6, 1.4],
    fill_mode='reflect',
    validation_split=0.15
)
```

Listing 2: Configuración de aumento de datos

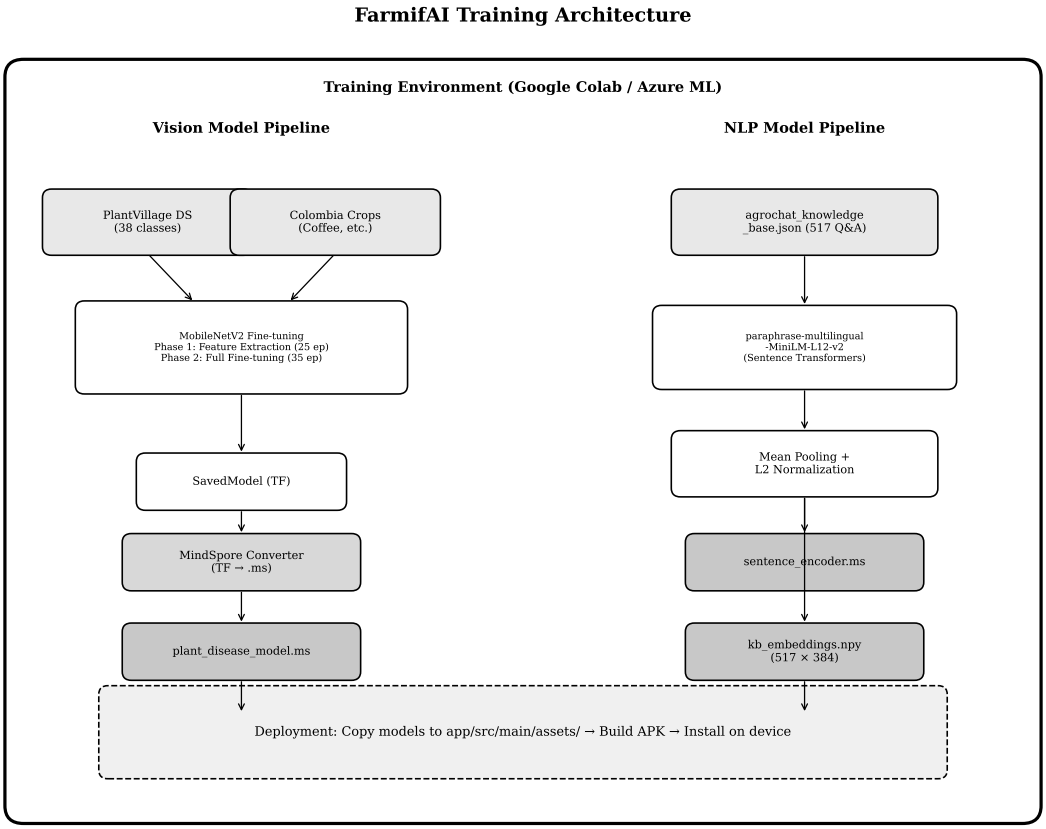

Figura 3: Arquitectura completa de entrenamiento para modelos de visión y NLP

3.3. Pipeline de Inferencia

La inferencia en dispositivo sigue el pipeline mostrado en la Figura **??**:

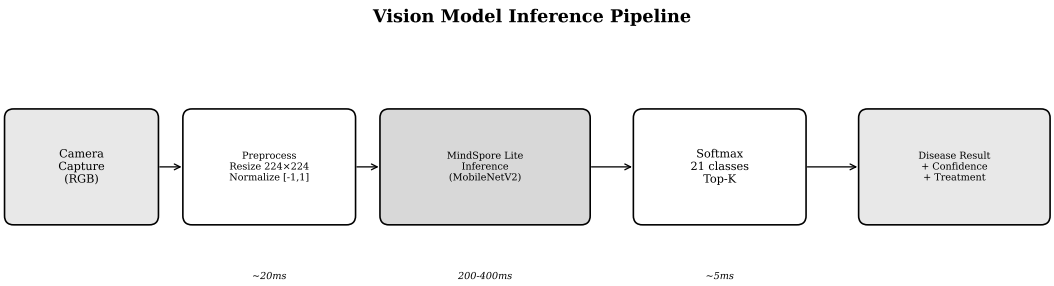

Figura 4: Pipeline de inferencia del modelo de visión en dispositivo Android

```
fun preprocessImage(bitmap: Bitmap): ByteBuffer {
    val scaled = Bitmap.createScaledBitmap(bitmap, 224, 224, true)

    // Asignar buffer: 224 * 224 * 3 canales * 4 bytes
    val buffer = ByteBuffer.allocateDirect(224 * 224 * 3 * 4)
    buffer.order(ByteOrder.nativeOrder())

```

```
    for (y in 0 until 224) {
        for (x in 0 until 224) {
            val pixel = scaled.getPixel(x, y)

            // Normalizar al rango [-1, 1]
            buffer.putFloat(((pixel shr 16 and 0xFF) - 127.5f) / 127.5f)
            buffer.putFloat(((pixel shr 8 and 0xFF) - 127.5f) / 127.5f)
            buffer.putFloat(((pixel and 0xFF) - 127.5f) / 127.5f)
        }
    }
    return buffer
}
```

Listing 3: Preprocesamiento de imagen para inferencia con MindSpore Lite

3.4. Despliegue

Conversión de modelo de TensorFlow a formato MindSpore Lite:

```
# Exportar SavedModel de TensorFlow
python tools/export_trained_model.py \
  --model_path outputs/best_model.h5 \
  --output_dir outputs/exported/

# Convertir a formato MindSpore Lite
python -m mindspore_lite.converter \
  --fmk=TF \
  --modelFile=outputs/exported/saved_model.pb \
  --outputFile=plant_disease_model \
  --inputShape=1,224,224,3

# Desplegar a assets de la app
cp plant_disease_model.ms app/src/main/assets/
```

Listing 4: Comandos de conversión y despliegue de modelo

4. Modelo NLP: Búsqueda Semántica

4.1. Implementación del Modelo

El componente de búsqueda semántica usa el modelo `paraphrase-multilingual-MiniLM-L12-v2` de Sentence Transformers para codificar tanto consultas como entradas de la base de conocimiento en embeddings de 384 dimensiones.

4.2. Generación de Embeddings

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

def encode_with_mean_pooling(model, tokenizer, texts):
    inputs = tokenizer(
        texts, padding="max_length", truncation=True,
        max_length=128, return_tensors="pt"
    )

    with torch.no_grad():
        outputs = model(**inputs)
        attention_mask = inputs['attention_mask']
        token_embeddings = outputs.last_hidden_state
```

```python

        # Expandir mascara para broadcasting
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(
            token_embeddings.size()
        ).float()

        # Mean pooling
        sum_embeddings = torch.sum(
            token_embeddings * input_mask_expanded, 1
        )
        sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
        embeddings = sum_embeddings / sum_mask

        # Normalizacion L2
        embeddings = F.normalize(embeddings, p=2, dim=1)

    return embeddings.numpy()
```

Listing 5: Mean pooling para embeddings de oraciones

4.3. Pipeline de Procesamiento de Consultas

pipeline de búsqueda semántica (Figura **??**) procesa consultas de usuarios de la siguiente manera:

1. Tokenizar consulta de entrada (máximo 128 tokens)
2. Codificar consulta usando codificador MiniLM
3. Calcular similitud coseno contra 517 embeddings pre-calculados de KB
4. Recuperar top-3 contextos sobre umbral (0.4)
5. Aumentar prompt con contextos recuperados
6. Generar respuesta usando Llama 3.2

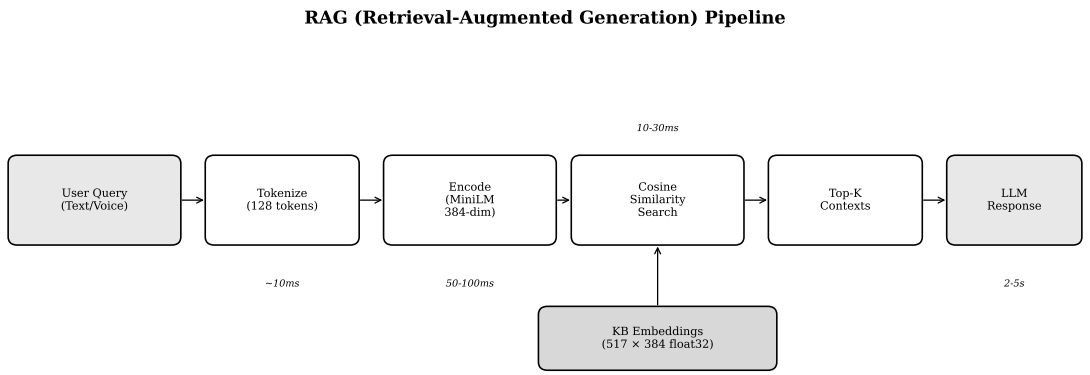

Figura 5: Pipeline de búsqueda semántica para generación de respuestas contextuales

```java
fun findTopKContexts(query: String, topK: Int = 3): List<Context> {
    val queryEmbedding = computeEmbedding(query)

    return kbEmbeddings.mapIndexed { i, emb ->
```

```
        val score = cosineSimilarity(queryEmbedding, emb)
        i to score
    }.filter { it.second >= 0.4f }
     .sortedByDescending { it.second }
     .take(topK)
     .map { (idx, score) ->
         Context(
             answer = kbEntries[idx].answer,
             score = score,
             category = kbEntries[idx].category
         )
     }
}

fun cosineSimilarity(a: FloatArray, b: FloatArray): Float {
    // Vectores normalizados L2, producto punto = similitud coseno
    return a.indices.sumOf { (a[it] * b[it]).toDouble() }.toFloat()
}
```

Listing 6: Implementación de búsqueda por similitud coseno

5. LLM: Modelo de Lenguaje Local

5.1. Implementación del Modelo

FarmifAI usa Llama 3.2 1B Instruct con cuantización Q4_K_M, reduciendo el tamaño del modelo de ∼2GB a ∼750MB manteniendo la calidad de las respuestas.

5.2. Inferencia

```
fun buildPrompt(context: String, query: String): String {
    val systemPrompt = """Eres FarmifAI, un asistente
        agricola experto. Responde en espanol.
        Usa el contexto proporcionado para responder."""

    return """<|begin_of_text|><|start_header_id|>system<|end_header_id|>

$systemPrompt<|eot_id|><|start_header_id|>user<|end_header_id|>

Contexto: $context
Pregunta: $query<|eot_id|><|start_header_id|>assistant<|end_header_id|>

"""
}

suspend fun generate(prompt: String, maxTokens: Int = 150): Flow<String> = flow
    {
    llama.setTemperature(0.7f)
    llama.setTopP(0.9f)
    llama.generate(prompt).collect { token ->
        if (!token.contains("<|eot_id|>")) emit(token)
    }
}
```

Listing 7: Construcción de prompt y generación con Llama 3.2

5.3. Despliegue

```
# Descargar modelo cuantizado
wget https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF/\
resolve/main/Llama-3.2-1B-Instruct-Q4_K_M.gguf

# Desplegar al almacenamiento externo del dispositivo
adb shell mkdir -p /sdcard/Android/data/edu.unicauca.app.agrochat/files/
adb push Llama-3.2-1B-Instruct-Q4_K_M.gguf \
  /sdcard/Android/data/edu.unicauca.app.agrochat/files/
```

Listing 8: Despliegue del LLM al dispositivo Android

6. Interfaz de Voz

6.1. Voz a Texto (Vosk)

```
fun initializeVosk() {
    val modelDir = File(context.cacheDir, "model-es-small")
    if (!modelDir.exists()) {
        unpackAssets("model-es-small", modelDir)
    }
    voskModel = Model(modelDir.absolutePath)
}

fun startListening() {
    val recognizer = Recognizer(voskModel, 16000.0f)
    speechService = SpeechService(recognizer, 16000.0f)
    speechService?.startListening(object : RecognitionListener {
        override fun onResult(hypothesis: String?) {
            hypothesis?.let {
                val json = JSONObject(it)
                val text = json.getString("text")
                if (text.isNotBlank()) onResult?.invoke(text)
            }
        }
    })
}
```

Listing 9: Integración de Vosk STT

7. Resultados y Evaluación

7.1. Métricas de Rendimiento

La Tabla **??** muestra las métricas de rendimiento para cada componente.

Cuadro 2: Métricas de rendimiento en dispositivo Android de gama media (Snapdragon 6xx)

Componente	**Latencia**	**Memoria**	**Precisión**
Modelo de Visión	200-400ms	50MB	92.3 % Top-1
Búsqueda Semántica	60-130ms	100MB	94.4 % Top-1
Generación LLM	2-5s	1.2GB	–
Voz STT	Tiempo real	100MB	85-92 %

7.2. Resultados de Detección de Enfermedades

El modelo de visión logra:

- **Precisión Top-1**: 92.3 %
- **Precisión Top-3**: 98.1 %
- **Tiempo de Inferencia**: 200-400ms

8. Flujo de Trabajo de Despliegue

La Figura **??** ilustra el flujo de trabajo completo de despliegue desde el entrenamiento del modelo hasta la instalación en el dispositivo.

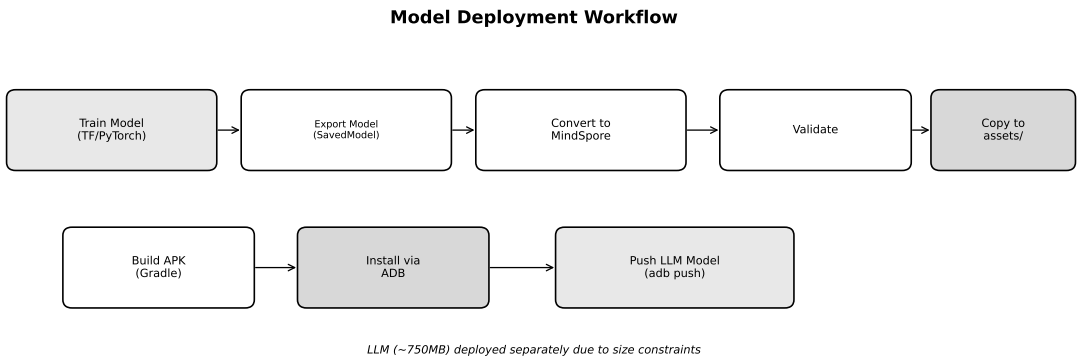

Figura 6: Flujo de trabajo de despliegue de modelos

9. Datasets y Fuentes de Datos

9.1. Datos de Entrenamiento del Modelo de Visión

El modelo de clasificación de enfermedades fue entrenado usando dos datasets complementarios:

Dataset PlantVillage: Un dataset públicamente disponible que contiene 54,309 imágenes de hojas de plantas sanas y enfermas en 14 especies de cultivos. Extrajimos imágenes para nuestros cultivos objetivo (tomate, papa, pimiento, maíz) cubriendo 18 clases de enfermedades más estados saludables. Disponible en: `https://github.com/spMohanty/PlantVillage-Dataset` y `https://www.kaggle.com/datasets/emmarex/plantdisease`.

Dataset BRACOL de Café: El dataset Brazilian Arabica Coffee Leaf (BRACOL) contiene 1,747 imágenes de hojas de café arábica afectadas por los principales estreses bióticos. El dataset incluye cuatro clases de enfermedades:

- Minador de la hoja
- Roya del café
- Mancha parda
- Mancha de Cercospora

Disponible en: `https://data.mendeley.com/datasets/yy2k5y8mxg/1` (Paper: `https://doi.org/10.1016/j.compag.2019.105162`).

Ambos datasets fueron preprocesados a 224×224 píxeles y aumentados usando rotaciones, volteos, zooms y variaciones de brillo para aumentar la robustez del modelo.

9.2. Base de Conocimiento NLP

El componente de búsqueda semántica usa una base de conocimiento agrícola curada que consiste en:

- 517 pares pregunta-respuesta cubriendo manejo de cultivos, identificación de enfermedades, control de plagas y mejores prácticas
- Preguntas obtenidas de servicios de extensión agrícola, foros de agricultores y consultas con expertos
- Contenido validado por agrónomos de la Universidad del Cauca
- Categorías: Manejo de Cultivos (35 %), Control de Enfermedades (30 %), Manejo de Plagas (20 %), Suelo y Fertilización (15 %)

Cada entrada incluye múltiples variaciones de preguntas para mejorar la coincidencia semántica, con embeddings pre-calculados usando el modelo paraphrase-multilingual-MiniLM-L12-v2.

9.3. Modelo de Reconocimiento de Voz

El reconocimiento de voz offline usa el modelo de idioma español pre-entrenado de Vosk (`vosk-model-small-es-0.42`):

- Datos de entrenamiento: Corpus Common Voice en español + datasets propietarios
- Tamaño del modelo: 50MB comprimido
- Optimizado para dialectos del español latinoamericano
- Vocabulario adaptado incluye terminología agrícola

Disponible en: `https://alphacephei.com/vosk/models/vosk-model-small-es-0.42.zip` (Modelos oficiales: `https://alphacephei.com/vosk/models`).

9.4. Modelo de Lenguaje

El modelo Llama 3.2 1B Instruct fue desarrollado por Meta AI:

- Pre-entrenado en texto diverso de internet y código
- Ajustado para tareas de seguimiento de instrucciones
- Cuantizado a formato Q4_K_M usando llama.cpp
- No se aplicó ajuste fino adicional específico del dominio

Disponible en: `https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct` (Cuantizado: `https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF`).